

IMAGE SEGMENTATION AND MORPHOLOGICAL IMAGE PROCESSING

AUDIO PROCESSING, VIDEO PROCESSING AND COMPUTER VISION

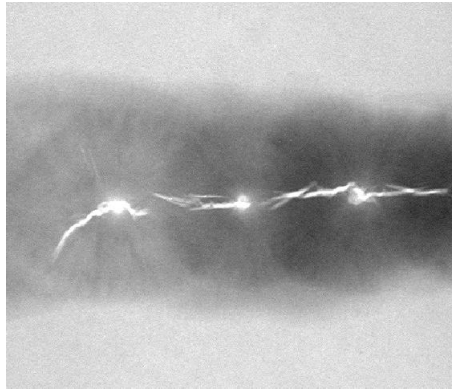
LAB EXERCISE 3

BACHELOR IN DATA SCIENCE AND ENGINEERING

1.- IMAGE SEGMENTATION

1.1.- THRESHOLD-BASED SEGMENTATION

Initially work with image 'Im1'. It is an X-ray image of a weld in which the brightest interior regions correspond to defects of this weld that let X-rays pass without attenuation.



- Segment the image by means of a gray-level threshold; for doing this, use the functions `skimage.filters.threshold_otsu` (that computes the Otsu's threshold) and then apply this threshold to obtain a binary image. Visualize the result.
- Obtain and represent the histogram of the original image. Determine a more adequate threshold to segment the defective interior regions. Segment the image using this new threshold.
- Segment again the image by applying a threshold over a previously low-pass filtered image. Use a Gaussian filter.

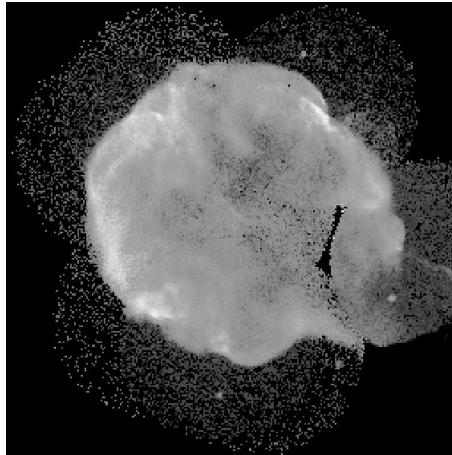
1.2.- GRAYLEVEL IMAGE CLUSTERING-BASED SEGMENTATION

Work with image 'Im2'.



- Use the k-means algorithm (`sklearn.cluster.KMeans`) to cluster the image pixels according to their gray level (use 3 centroids).
- Display the histogram of gray levels of the image. Display the obtained centroids in the same figure.
- Display, using pseudocolor, the segmented image.

Work with the image 'Im3'.



- Perform a clustering-based segmentation of the image according to the gray level (consider three classes again). The goal is to segment the “ring” of less the matter, the dense inner region and the background.
- Display the segmented image.
- Display the histogram and the obtained centroids.
- Repeat the exercise, but now rely on a local average image instead of the original one. Use the function `scipy.ndimage.uniform_filter` to obtain the local mean.
- Combine mean and variance for carrying out the segmentation. Use `scipy.ndimage.generic_filter` to compute the variance image
- Display the segmented image.
- Display, now in two dimensions, the data and centroids.

1.3.- GRAYLEVEL IMAGE CLUSTERING-BASED SEGMENTATION

The goal of this exercise is to explore how K-means clustering can be used to segment a color image. Each pixel will be represented by its RGB values and grouped into one of five clusters according to color similarity. The resulting segmentation will then be visualized both in the image domain and in the three-dimensional RGB feature space.

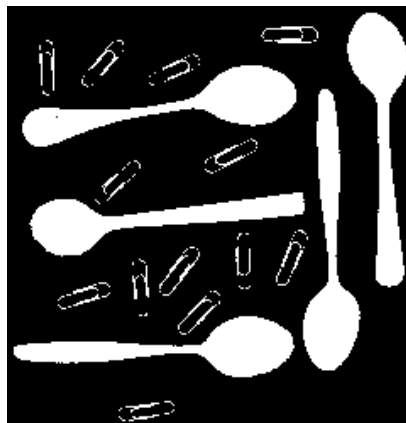
Work with the image 'peppers.png'.



- Read the color image and display it. Optionally reduce the image size to make the clustering faster.
- Represent each pixel by its three RGB values, so that every pixel becomes a point in a three-dimensional feature space.
- Apply K-means clustering using 5 clusters intended to capture meaningful color groups such as red, green, yellow, white, and background.
- Reconstruct the segmented image by replacing each pixel with the RGB value of the centroid of its cluster.
- Display the original image and the segmented image side by side.
- Plot the pixels in the 3D RGB feature space and indicate the five cluster centroids. Observe whether the clusters approximately correspond to intended color groups.

2.- MORPHOLOGICAL IMAGE PROCESSING

Work with the image `spoons_binary.png`.



2.1.- REMOVAL OF NON-HORIZONTAL OBJECTS

- Define a horizontal rectangular structuring element using `skimage.morphology.rectangle` such that an opening operation (`skimage.morphology.binary_opening`) removes all objects except for the horizontal spoons. Apply the opening operation and display the result. Notice that some parts of the horizontal objects that remain after the opening operation have also been eroded. The goal of the next exercise is to reconstruct these missing regions.

2.2.- RECONSTRUCTION OF THE HORIZONTAL SPOONS

- Define a function called `dilate_mask` that dilates the image obtained in the previous exercise using `skimage.morphology.binary_dilation` with a diamond-shaped structuring element (`skimage.morphology.diamond`) of size 1. Within the same function, compute a logical AND operation (`np.logical_and`) between the dilated image and the original image. This constrains the dilation so that new pixels can only be added where they are present in the original image.
- Apply this function iteratively, using the output of each iteration as the input to the next one. Observe how the missing regions are gradually reconstructed.